



Trabalho da disciplina – Roteiro e template

Profs. Drs. Mauro Spinola e Marcelo Pessôa. Monitores Caio Azevedo e Priscila Bayer

Versão 2 (27-05-2026)

Sumário

1	Caracterização do projeto (Contexto de negócio)	3
1.1	Identificação do Projeto	3
1.2	Organização Cliente	3
1.3	Problema identificado	3
1.4	Objetivo do software	3
1.5	Público-alvo / Usuários	4
2	Escopo do sistema (delimitação)	4
2.1	Escopo incluído	4
2.2	Fora do escopo (opcional)	4
3	Requisitos do sistema	5
3.1	Matriz geral de requisitos	5
3.2	Requisitos funcionais	5
3.3	Requisitos não funcionais	6
4	Prototipação da solução	6
4.1	Objetivos do protótipo	6
4.2	Protótipo de baixa fidelidade	7
4.3	Protótipo de alta fidelidade ou funcional	7
4.4	Aprendizados obtidos com o protótipo	8
5	Conclusão do Ciclo 1	8
6	Arquitetura e implementação do Backend	8
6.1	Estrutura de pastas e separação de responsabilidades	8
6.2	Contrato da API (Endpoints e Lógica)	8
6.3	Documentação e Integração com Frontend	9
7	Infraestrutura e containerização (Docker)	9
7.1	Receitas de construção (Dockerfiles)	9
7.2	Orquestração de serviços (Docker Compose)	9



PRO3151 Laboratório de Sistemas de Informação

7.3	Padronização e variáveis de ambiente	9
8	Camada de persistência (Banco de Dados)	9
8.1	Modelo relacional e esquema de tabelas	9
8.2	Integridade de dados e restrições (constraints)	10
8.3	Consultas de negócio e análise de dados	10
9	Conclusão do Ciclo 2	10
10	Conclusão do Trabalho	11



1 Caracterização do projeto (Contexto de negócio)

1.1 Identificação do Projeto

- Nome do projeto
- Grupo / Integrantes (nome e NUSP)
- Data da entrega
- Versão do documento

1.2 Organização Cliente

Objetivo: **entender o contexto em que o software será utilizado.**

Descrever brevemente a organização ou pessoa que será atendida pelo sistema:

- Nome da organização ou cliente
- Área de atuação
- Principais produtos ou serviços
- Breve descrição de como o trabalho é realizado atualmente
- Papel do cliente no projeto (usuário final, patrocinador etc.)

1.3 Problema identificado

Descrever **qual problema real motivou o desenvolvimento do sistema.**

Sugestões:

- Situação atual (como o processo ocorre hoje)
- Principais dificuldades ou limitações. "Dor" real observada.
- Impactos do problema (tempo, custo, erros, falta de informação etc.)

Boas perguntas orientadoras:

- O que não funciona bem hoje ou poderia ser melhorado?
- Quem sofre com o problema?
- Por que vale a pena resolver esse problema?

1.4 Objetivo do software

Descrever **o que o software pretende fazer para resolver o problema identificado.**

Sugestões:

- Finalidade principal do sistema
- Benefícios esperados
- Principais funções que ele deverá oferecer



Exemplo de formulação:

O objetivo do sistema é apoiar a gestão de pedidos de uma pequena confeitaria, permitindo registrar pedidos, acompanhar produção e organizar entregas.

1.5 Público-alvo / Usuários

Descrever **quem utilizará o sistema**.

Exemplos:

- Funcionários da empresa
- Clientes
- Administradores
- Parceiros

Se possível, indicar **tipos de usuário (papéis)**.

Exemplo:

- Administrador
- Usuário operacional
- Cliente

2 Escopo do sistema (delimitação)

Descrever **o que o sistema inclui e o que está fora do escopo**.

Isso ajuda a **limitar o projeto e evitar expectativas irreais**.

2.1 Escopo incluído

Exemplos:

- cadastro de usuários
- registro de pedidos
- consulta de informações
- geração de relatórios simples

2.2 Fora do escopo (opcional)

Exemplos:

- integração com sistemas externos
- pagamentos online
- análise avançada de dados



3 Requisitos do sistema

Requisitos descrevem o que o sistema deve fazer e como deve se comportar.

Dividem-se em:

- requisitos funcionais
- requisitos não funcionais

3.1 Matriz geral de requisitos

Apresentar uma visão resumida dos requisitos identificados.

Exemplo:

Código	Nome	Tipo (F/NF)	Descrição resumida
REQ01	Cadastrar usuário	F	Permite cadastro de usuários
REQ02	Consultar pedidos	F	Permite consulta de pedidos
REQ03	Segurança de acesso	NF	Acesso restrito por autenticação

3.2 Requisitos funcionais

Os requisitos funcionais descrevem as funcionalidades do sistema.

Cada requisito deve conter:

- Código
- Nome
- Descrição (Explicação clara usando verbos de ação)
- Critérios de aceitação (Condições que devem ser atendidas para que o requisito seja considerado concluído)
- **Modelo:**

Código: REQ01

Nome: Cadastrar usuário

Descrição:

O sistema deve permitir o cadastro de novos usuários contendo nome, e-mail e senha.

Critérios de aceitação:

- O sistema deve impedir cadastro com e-mail já existente
- O sistema deve exigir senha com mínimo de 6 caracteres

Sugestão: cada grupo deve apresentar **entre 5 e 10 requisitos funcionais principais**.



3.3 Requisitos não funcionais

Requisitos não funcionais descrevem **características de qualidade ou restrições do sistema**.

Importante: Devem ser mensuráveis (ex: "tempo de resposta < 2s" em vez de "o sistema deve ser rápido")

Exemplos de categorias:

- desempenho
- segurança
- usabilidade
- disponibilidade
- compatibilidade
- manutenibilidade
- **Modelo:**

Código: REQNF01

Nome: Tempo de resposta

Descrição:

O sistema deve responder às requisições do usuário em até 2 segundos.

Critério de aceitação:

95% das requisições devem ser respondidas em menos de 2 segundos.

Sugestão: cada grupo deve apresentar **entre 3 e 6 requisitos não funcionais**.

4 Prototipação da solução

A prototipação consiste em **criar uma versão inicial do sistema para explorar ideias e validar requisitos antes da implementação completa**.

Ela ajuda a reduzir ambiguidades e melhorar a comunicação entre desenvolvedores e usuários.

4.1 Objetivos do protótipo

Descrever:

- o que se pretende avaliar com o protótipo
- quais aspectos da solução estão sendo explorados

Exemplos:

- fluxo de uso do sistema
- estrutura das telas
- interação do usuário



PRO3151 Laboratório de Sistemas de Informação

Sugestões para as apresentações dos protótipos de baixa e alta fidelidade (itens 4.2 e 4.3):

Apresentar os **principais modelos de interface do sistema**.

Sugestão de telas:

- tela inicial
- tela de login
- tela principal do sistema
- telas das funcionalidades principais

Para cada tela, incluir:

- imagem ou diagrama
- breve explicação de sua função

Aspectos chave da UX:

- **Simplicidade e intuição:** Demonstre como a interface reduz a curva de aprendizado do usuário.
- **Portabilidade:** Como a interface se adapta a diferentes dispositivos (Mobile, Web, Desktop)

4.2 Protótipo de baixa fidelidade

- esboços de telas
- wireframes simples

4.3 Protótipo de alta fidelidade ou funcional

- telas próximas ao sistema final
- interface navegável
- pequena aplicação executável

Incluir descrição de **como o usuário interage com o sistema**.

Exemplo:

1. Usuário acessa o sistema
2. Realiza login
3. Consulta informações
4. Realiza operação principal
5. Encerra sessão

Se desejado, pode-se incluir:

- **fluxograma**



- diagrama simples de navegação entre telas.

4.4 Aprendizados obtidos com o protótipo

Breve reflexão:

- O que foi aprendido com a prototipação?
- Quais requisitos foram refinados?
- Que melhorias foram identificadas?
- **Refinamento de Requisitos:** Descreva como o feedback do protótipo ajudou a transformar requisitos vagos em critérios testáveis (ex: após o teste, percebemos que o fluxo de compra precisava ter no máximo 4 passos).

5 Conclusão do Ciclo 1

Apresentar uma síntese do trabalho realizado.

Sugestões:

- visão geral da solução proposta
- principais funcionalidades do sistema
- próximos passos do projeto

6 Arquitetura e implementação do Backend

6.1 Estrutura de pastas e separação de responsabilidades

Apresentar a organização física do código (ex: pastas /backend e /frontend) e justificar por que esses componentes rodam em processos separados.

6.2 Contrato da API (Endpoints e Lógica)

Listar os principais endpoints desenvolvidos no FastAPI.

Modelo: Contrato da API (Exemplo FastAPI)

Nesta tabela, o grupo deve listar os caminhos que o frontend utiliza para "conversar" com o backend.

Método HTTP	Rota (Endpoint)	Descrição da Funcionalidade	Payload (Entrada)	Resposta Esperada (Sucesso)
POST	/talhoes/	Cadastra uma nova área de plantio no sistema.	{"nome": string, "area": float}	{"status": "sucesso", "id": 101}
GET	/talhoes/{id}	Busca os detalhes de um talhão específico pelo ID.	Parâmetro de URL: id	{"id": 101, "nome": "Talhão A"}



Método HTTP	Rota (Endpoint)	Descrição da Funcionalidade	Payload (Entrada)	Resposta Esperada (Sucesso)
DELETE	/talhoes/{id}	Remove um talhão da base de dados.	Parâmetro de URL: id	{"mensagem": "Removido"}

6.3 Documentação e Integração com Frontend

Descrever como o uso da documentação automática (/docs) auxiliou o desenvolvimento do frontend em Streamlit e como o "contrato" da API garantiu que a interface consumisse os dados corretamente.

7 Infraestrutura e containerização (Docker)

7.1 Receitas de construção (Dockerfiles)

Descrever a criação das imagens para o backend e frontend.

- **Elementos chave:** Imagem base (ex: python:3.12-slim), diretório de trabalho, instalação de dependências e comando de inicialização.

7.2 Orquestração de serviços (Docker Compose)

Explicar como o arquivo docker-compose.yml coordena o backend, o frontend e o banco de dados PostgreSQL para subirem de forma integrada.

- **Destaque:** Descrever a **Rede Docker Interna** que permite aos serviços se comunicarem pelo nome (ex: http://backend:8000).

7.3 Padronização e variáveis de ambiente

Explicar o uso de **variáveis de ambiente** para configurar chaves de API e credenciais do banco, garantindo segurança e flexibilidade entre ambientes local e nuvem

8 Camada de persistência (Banco de Dados)

8.1 Modelo relacional e esquema de tabelas

Apresentar o diagrama ou script SQL das tabelas principais (ex: usuarios, transacoes, analise_ia).

- Identificar claramente as **Chaves Primárias (PK)** e **Chaves Estrangeiras (FK)**.

(Lembre-se que a **Documentação Automática (/docs)** do FastAPI pode ser usada como fonte para o contrato da API, e o **DBeaver** fornece a visualização de colunas e constraints necessária para o modelo relacional.)

Modelos:

Tabela: Usuário (Exemplo)



PRO3151 Laboratório de Sistemas de Informação

Coluna	Tipo de Dado	Restrições (Constraints)	Descrição
user_id	SERIAL / INT	PK (Primary Key), NOT NULL	Identificador único e imutável do usuário.
nome	VARCHAR(100)	NOT NULL	Nome completo do usuário; a restrição garante que o campo não seja vazio.
cpf	VARCHAR(11)	UNIQUE, NOT NULL	CPF único para evitar cadastros duplicados, mantendo a integridade dos dados.
email	VARCHAR(150)	UNIQUE	Endereço de e-mail para login, com garantia de exclusividade no banco.

Tabela: Pedido (Exemplo de Relacionamento)

Coluna	Tipo de Dado	Restrições (Constraints)	Descrição
order_id	INT	PK, NOT NULL	Identificador único e obrigatório de cada pedido, garantindo a identidade de cada registro.
user_id	INT	FK (Ref. usuarios.user_id)	Chave estrangeira que estabelece a integridade referencial, conectando o pedido a um usuário existente na tabela de usuários.
valor	NUMERIC	CHECK (valor > 0)	Regra de validação no banco que garante que não existam registros com valores negativos ou zerados, protegendo a lógica de negócio.

8.2 Integridade de dados e restrições (constraints)

Descrever quais regras de negócio foram movidas para o banco de dados através de *constraints* como NOT NULL, UNIQUE e integridade referencial.

- **Exemplo:** Como o banco impede o cadastro de dois usuários com o mesmo CPF.

8.3 Consultas de negócio e análise de dados

Apresentar pelo menos 3 consultas SQL (utilizando SELECT, GROUP BY e JOIN) que extraiam inteligência do sistema para o usuário final.

9 Conclusão do Ciclo 2

Apresentar uma síntese da evolução do sistema.

- **Visão Geral:** Como o sistema deixou de ser um protótipo volátil para se tornar uma aplicação profissional persistente.



PRO3151 Laboratório de Sistemas de Informação

- **Sobrevivência:** Como o Docker e o PostgreSQL garantem os requisitos não funcionais especificados (confiabilidade, escalabilidade...)
- **Próximos passos:** Melhorias futuras.

10 Conclusão do Trabalho

- **Análise dos resultados:** análise do produto desenvolvido
- **Aprendizado:** discussão sobre o aprendizado obtido
- **Disciplina:** avaliação e sugestões de aprimoramento.